ENTERPRISE ARCHITECTURE / BANKING / APPLIED AI

From "I've Lost My Debit Card" to an Intelligent Banking Journey

Designing agentic AI for debit-card servicing, origination, contextual memory, and adaptive customer experiences

A customer opens their bank's mobile app and types:

> *"I've lost my card. This is the third time I've had to contact you. I'm travelling tomorrow, and I really need this sorted."*

A conventional chatbot might return a link to the "Lost or stolen card" page. A better digital assistant might explain how to block the card. But what would a truly *journey-aware* banking experience do?

It would recognise an urgent card-servicing request, acknowledge the customer's frustration without pretending to read their mind, confirm which card is affected, check what has already happened, enforce the appropriate security controls, carry out the permitted actions, and adapt the screen to show only what matters next.

If the customer switches to the contact centre, the conversation should not start again from zero.

That is the business problem I want to explore here: **How do we architect an agentic banking application that is context-aware, secure, economical with LLM calls, and capable of dynamically adapting its customer experience?**

The example is debit-card servicing and origination, but the patterns extend to disputes, onboarding, lending enquiries, and many other regulated workflows. The architecture below is a conceptual reference design, not a representation of any particular bank's internal systems.

1. Start with the business requirement, not the LLM

Imagine we have been asked to modernise three common journeys:

- **Lost or stolen debit card:** identify the card, protect the account, block the affected card when authorised, arrange replacement where eligible, and explain what happens next.
- **Damaged debit card:** check whether replacement is appropriate, confirm delivery details, initiate the correct replacement flow, and communicate fulfilment status.
- **New debit card / card origination:** establish the customer's account and eligibility, identify the correct product and required checks, obtain the necessary approvals and confirmations, then initiate card issuance.

The customer might begin on mobile, the web, text chat, voice/IVR, or through a contact-centre representative. They may abandon the journey, return tomorrow, change channels, or ask a related question halfway through.

Success is not merely "the AI answered correctly." Success means the bank can deliver **a completed, explainable, resumable and safe business outcome**.

To get there, we need several layers working together: channel processing, security, durable orchestration, contextual memory, specialist agents, bank-owned systems of record, and an adaptive experience layer.

2. One customer request can arrive in five very different forms

Consider the same request across channels:

- **Mobile app:** typed text, button selections, authentication state, device and session metadata.
- **Web banking:** chat messages, form entries, browser context and current page or journey state.
- **Contact centre:** representative-entered notes, verified caller context and, where permitted, a call transcript.
- **Chat:** conversation turns, selected options, prior assistant responses and customer corrections.
- **Voice:** audio that may first pass through speech-to-text, language detection and transcription-quality checks.

Before orchestration, we need a **channel normalisation and input-understanding capability** that converts these inputs into a consistent request envelope: customer/session reference, source channel, message or transcript, conversation ID, language, timestamps, and relevant verified metadata.

A useful distinction: **channel metadata is observed; customer intent and sentiment are inferred**. They should not appear as trusted facts supplied by the app.

And the diagram's logical ordering is not necessarily the physical security ordering. In production, the network edge, authentication, authorisation, abuse controls and approved handling of sensitive content must be enforced before data reaches protected model-processing components.

3. Where do intent and sentiment actually come from?

The words "lost card" do not magically become an `intent=LOST_CARD` field.

An **intent-classification component** has to infer the customer's goal. This can be implemented with:

1. Deterministic routing for explicit actions, such as a customer selecting "Report a lost card."
2. A smaller trained NLP classifier for familiar utterances.
3. An LLM with structured output for complex, multi-intent or ambiguous requests.

A possible result is:

```
intent = REPORT_LOST_CARD; urgency = HIGH; clarification_needed = TRUE; confidence = 0.91
```

That score is illustrative, not a guarantee. If the customer says, "My card isn't working and I can't find it," the correct next step may be clarification rather than immediately treating the card as stolen.

Sentiment or emotion detection is a different operation. A model may observe language consistent with frustration, confusion, urgency or satisfaction. With voice, acoustical signals may sometimes help, but these signals are imperfect and culturally variable. A bank should not assert that it *knows* how a customer feels.

The practical purpose is modest and valuable: choose an appropriate communication style. For a potentially distressed customer, use concise reassurance, visible next steps and fewer distractions. For a curious customer, provide clear exploration options. **Do not use inferred vulnerability or frustration to pressure customers into products or offers.** Customers should be able to correct the assistant and request a human at any stage.

4. The security layer: not every intelligent operation needs an LLM

Once the request enters the bank's trusted backend, familiar controls still matter: authentication, authorisation, throttling, fraud and bot protections, request validation, secure logging and channel-specific policies.

There is also a vital question: **How do we prevent sensitive customer data from being sprayed across prompts, traces, tools and third-party systems?**

Introduce a dedicated **PII/data-protection service**, invoked at the relevant trust boundaries. It can detect and protect names, phone numbers, addresses, account identifiers, payment-card data and other sensitive fields.

That service does *not* need a generative LLM for every message. Real implementations can combine regular expressions, checksum and format validation, banking-specific recognisers, traditional named-entity recognition, and optional LLM assistance for ambiguous cases. Microsoft Presidio and Azure AI Language PII detection are examples of this broader class of technology.

A critical nuance: sending unredacted text to an external LLM *to discover whether it contains sensitive data* can itself create a disclosure problem. The detector, its processing environment, data retention and downstream recipients must be approved.

Protection must recur **before model calls, around tool inputs and outputs, during retrieval, and before responses or logs are stored**. A single ingress scan cannot guarantee that future agents will never see sensitive information.

5. The journey orchestrator is the conductor, not another chatbot

Here is where agentic AI becomes an architectural discipline rather than a collection of prompts.

The **journey orchestrator** owns the business workflow: which step is active, what has been completed, what approvals remain, which agent should work next, and how the journey resumes after a timeout or channel switch.

For a lost-card journey, the workflow might be:

```
START → identify intent → confirm affected card → verify required controls → block card → check replacement eligibility → confirm delivery → order replacement → notify → COMPLETE
```

Some stages involve an LLM; others should be deterministic rules or bank-owned APIs. A specialist agent can *recommend* the next action, but it should not be the authority for whether a card is blocked, an account is eligible or a regulated control is waived.

The orchestrator persists a **checkpoint** after important state transitions: workflow ID, customer reference, step name, version, decisions, pending tool calls and approval status. This supports retries, human review, idempotency and recovery.

Think of it this way: **the LLM proposes and explains; the workflow engine governs and records; the core banking system executes authoritative changes**.

6. Context is not the same thing as conversation history

One of the hardest questions in multi-agent systems is: *What should the next agent know?*

Suppose the customer spent 30 minutes chatting about travel before reporting a lost debit card. If we forward the entire transcript to every subsequent agent, cost and latency rise, and irrelevant details may distract the model. If we send only "lost card," we may omit the critical fact that the customer is travelling tomorrow.

I would separate information into three categories:

- **Conversation history:** the sequence of messages and events across channels.
- **Workflow state:** what the process has verified, decided, completed and still needs to do.
- **Agent context:** the minimal, permission-appropriate subset assembled for a specific task and a specific invocation.

For the card-service agent, a compact context packet might contain:

```
intent=REPORT_LOST_CARD; verified_customer_ref=tokenized-id; affected_card_ref=card-token;
status=active; urgency=travel_tomorrow; blocked=false; required_next_step=confirm_card
```

Notice what is missing: full card number, the customer's unrelated mortgage conversation, and assumptions about their personality. The next agent receives **task-relevant context**, not an indiscriminate data dump.

7. So where does "memory" actually live?

There is no single magical "agent memory database." Different types of memory answer different questions.

A. Workflow checkpoints — "Where are we in this journey?"

Use a durable transactional store such as PostgreSQL, a compatible managed relational database, or a carefully designed document store such as DynamoDB or Cosmos DB. Persist workflow versions, pending approvals, retries and resumption tokens. A cache alone is not sufficient for critical workflow durability.

B. Interaction history — "What happened, and what was said?"

Use governed transcript and event storage: relational metadata plus encrypted object storage where appropriate. Store channel, timestamp, consent and retention metadata. An append-only audit/event stream (for example Kafka or Kinesis plus an appropriate durable sink) can provide a trace of actions and outcomes. Audit retention and conversation retention need not be identical.

C. Structured customer memory — "What useful facts are we permitted to remember?"

This is usually **not a vector database**. It might be a relational table, JSON document or key–value/document record in PostgreSQL JSONB, DynamoDB, MongoDB or Cosmos DB. Examples include a confirmed communication preference, an explicitly saved accessibility preference, or the last successfully completed card-replacement journey. Each fact should have provenance, scope, expiry and a correction mechanism.

Do not silently turn an inferred emotion into a permanent customer profile. In many cases, temporary journey-level tone adaptation is the safer design.

D. Semantic search — "Which past event or policy is relevant to this question?"

A vector index (for example OpenSearch, Elasticsearch, pgvector, Pinecone or Weaviate) can help retrieve similar interactions and relevant knowledge. The vector is a *search representation*, not the customer record of truth. Retrieval must be permission-filtered, and results must be validated against current authoritative data.

E. Hot session cache — "What do we need in milliseconds?"

Redis or a comparable cache can hold short-lived, scoped context, temporary session state or UI metadata. Use TTLs and access controls. Never confuse a fast cache with durable card-servicing evidence.

F. Knowledge and policy store — "What are the bank's current rules?"

A versioned, approved repository can contain product documents, FAQs, servicing procedures and policy content. A retrieval service returns relevant, attributable snippets; a model does not invent the bank's rules.

The architecture works because these stores have **different responsibilities and trust levels**.

8. The context builder: retrieve, rank, summarise, filter and budget

This is the heart of efficient agent handoffs.

The **context builder** asks what the next agent needs, gathers candidates from authorised sources, ranks them, applies policy and redaction rules, summarises where appropriate, and emits a bounded context package.

A simple mental model is:

```
available_input_tokens = model_context_limit − reserved_output_tokens − fixed_instructions − safety_margin
```

Within that budget, give priority to the current customer request, verified security and workflow state, mandatory policy, relevant tool results, and only then useful historical context.

For example, an old 40-turn conversation may be condensed to: "Customer previously reported delivery issues; case X was resolved on date Y." But **summaries are lossy**. Never use a summary instead of checking the current card status or relying on a mandatory authorisation decision.

Summarisation is another point where an LLM *may* be called, typically asynchronously or when the transcript exceeds a threshold. You can also use extractive summarisation, structured event compression and cached summaries. Budgeting itself can be deterministic; a generative LLM is not required to subtract token counts.

Good context engineering is not about shrinking everything. It is about **retaining the information that changes the correct next action**.

9. Multiple agents, but not multiple sources of truth

The multi-agent layer can be organised around responsibilities:

- **Triage agent:** interpret the request, identify missing details and route the journey.
- **Card-service agent:** plan card-specific steps, request permitted tool calls and surface eligibility questions.
- **Risk and fraud capability:** combine deterministic policies, approved analytical models and specialist reasoning where useful; enforce step-up authentication and approvals.
- **Customer-communication agent:** explain actions in plain language, with a channel-appropriate tone and evidence-backed status.
- **Specialists:** handle identity, delivery-address updates, disputes or escalation when needed.

These are **logical roles**, not necessarily separate LLM deployments. Multiple agents may call the same model through a central model gateway, with different instructions and tools. Sometimes a single agent plus deterministic workflow is simpler and safer.

The model gateway is where you can see, control and meter actual LLM calls: intent classification when needed, selective summarisation, complex agent reasoning, grounded response generation, and optional UI schema generation. Put routing, model allowlists, timeout/retry policies, token quotas, safety checks, observability and cost accounting there.

The orchestration state should record *what the model suggested* separately from *what the bank verified and executed*.

10. The non-negotiable boundary: bank-owned systems of record

A card-service agent might call tools that wrap bank-owned APIs for card lookup, block/unblock, replacement issuance, delivery-address validation, notifications and CRM cases.

But a sentence such as “Your card has been blocked” must follow a **confirmed result from the card-management system**, not merely the LLM’s intention to call a tool.

For consequential actions, use explicit authorisation, customer confirmation where appropriate, least-privilege tool scopes, idempotency keys, secure API contracts and clear error recovery. If a request times out after the bank has already blocked the card, the next retry should *check the outcome* rather than blindly repeat the action.

The LLM is not the ledger. The bank’s systems of record remain the truth.

11. Dynamic UI/UX: reuse before you generate

Now for the customer-facing twist.

Why should everyone see the same static screen? A customer reporting a potentially stolen card may need a calm, focused interface with one prominent “Secure my card” action and a visible progress indicator. A customer exploring a new debit-card product may prefer comparison cards, eligibility guidance and a concise FAQ.

My preferred design is an **adaptive experience engine** backed by a **governed UI component and pattern library**.

Its decision sequence is:

1. Identify the journey stage, channel, accessibility needs and relevant interaction signals.
2. Retrieve a previously approved UI pattern, such as `card-loss-urgent-v3` or `replacement-address-confirmation-v2`.
3. Populate it with fresh, authorised customer and workflow data.
4. If nothing fits, optionally ask an LLM to propose a **declarative UI schema** using an allowlisted component vocabulary.
5. Validate accessibility, content, navigation, security, data bindings and brand rules before rendering.
6. Store only eligible, approved reusable patterns in the UI artefact registry; cache safe, non-personalised render metadata for later reuse.

Do not allow an LLM to generate arbitrary executable HTML/JavaScript and deploy it directly into an authenticated banking session. The model proposes within a controlled design system; the renderer and policy validators enforce the contract.

Equally important, the system should adapt **presentation**, not manipulate customers based on speculative emotional labels. For someone expressing frustration, the goal is fewer obstacles and a clearer route to resolution—not opportunistic selling.

12. The economics: where to spend tokens, and where not to

Calling a large model at every box is a surprisingly expensive way to build what could have been an ordinary workflow.

A cost-aware architecture uses a decision ladder:

- **Zero-model path:** structured buttons, known intents, standard policy checks, approved UI templates and deterministic API calls.
- **Small-model path:** ambiguous intent detection, language handling or limited classification.
- **Larger-model path:** complex dialogue, multi-intent reasoning, grounded explanation and genuinely novel UI schema proposals.
- **Offline path:** batch summarisation, knowledge indexing, UI-pattern evaluation and other work that need not delay the customer.

Measure token usage, model latency, retrieval latency, cache hit rate, cost per *completed* journey, escalation rates and customer effort. The most useful KPI is rarely “tokens saved” in isolation; it is **successful resolution at an acceptable cost and risk**.

A good adaptive UI should usually involve *retrieving and filling a vetted pattern*, not inventing a whole interface on every request.

13. Follow the lost-card journey from start to finish

Let's return to our frustrated traveller.

Step 1 — Ingress: The customer types the message in mobile banking. The trusted entry path checks session and request controls. The channel adapter provides a normalised envelope.

Step 2 — Understanding: A classifier or LLM identifies a likely lost-card request, possible urgency and language consistent with frustration. This is an inference, not a diagnosis. A PII protection service identifies sensitive fields and applies the appropriate handling rules.

Step 3 — Orchestration: The workflow engine opens or resumes `LOST_CARD`, records its version and determines the next required step.

Step 4 — Context assembly: It retrieves only relevant customer preferences, recent card-servicing events, approved policy and current card metadata from authorised sources. An old transcript is summarised only if useful; the token budgeter excludes irrelevant material.

Step 5 — Specialist execution: The card-service agent receives its bounded context packet. The risk gate enforces mandatory verification. After the authorised confirmation, the card API returns a definitive block result. A replacement workflow may then proceed according to eligibility and policy.

Step 6 — Adaptive response: The experience engine selects a reassuring, approved card-loss UI: confirmed status, immediate next action, replacement choices and delivery information. The communication agent produces a brief, grounded explanation.

Step 7 — Cross-channel continuation: The customer calls the contact centre. The representative sees the verified journey state, completed actions and relevant summary—not an uncontrolled dump of every chat message. The next step resumes from the checkpoint.

The impressive part is not that five agents spoke to each other. It is that **the customer did not have to reconstruct the story, the bank did not lose control of the transaction, and the interface reflected the situation**.

14. What could go wrong—and how would we know?

Before production, test the architecture against failure modes, not just happy-path demos:

- A customer mentions two cards, but the classifier chooses the wrong one.
- A transcript summary omits a critical customer correction.
- A vector search retrieves another customer's information or an outdated policy.
- A model produces a confident response despite a failed bank API call.
- A retry causes a duplicate replacement-card order.
- Emotion inference is wrong and produces a patronising interface.
- A dynamically proposed UI contains an unauthorised field or action.
- A workflow resumes after authentication or consent has expired.

Controls include tenant and customer isolation, permission-aware retrieval, confirmed-source checks, versioned policy, deterministic risk gates, human escalation, output validation, audit trails, prompt-injection defences, retention controls and end-to-end testing.

For Australian banking deployments, the relevant governance discussion should include APRA's CPS 234 information-security expectations, CPS 230 operational-risk requirements and applicable Australian Privacy Principles—not as a compliance label on the architecture, but as controls embedded in operating processes, vendor oversight and the data lifecycle.

15. Final thought: build an intelligent journey, not a clever chatbot

An agentic customer experience is not defined by how many LLM calls it makes, how many agents appear in its diagram or how personalised its opening greeting sounds.

It is defined by something more demanding: **the right information reaches the right component, at the right time, under the right authority, and produces the right customer outcome**.

The pattern I find compelling is straightforward:

Channels capture the request. Security establishes trust. Models help understand language. The orchestrator owns the journey. Memory preserves only what is useful and permitted. Agents perform bounded tasks. Bank APIs remain authoritative. The experience engine turns outcomes into a clear, responsive interface.

And wherever possible, **retrieve and reuse before you regenerate**—whether that means a context summary, a policy snippet, an interaction pattern or an approved UI component.

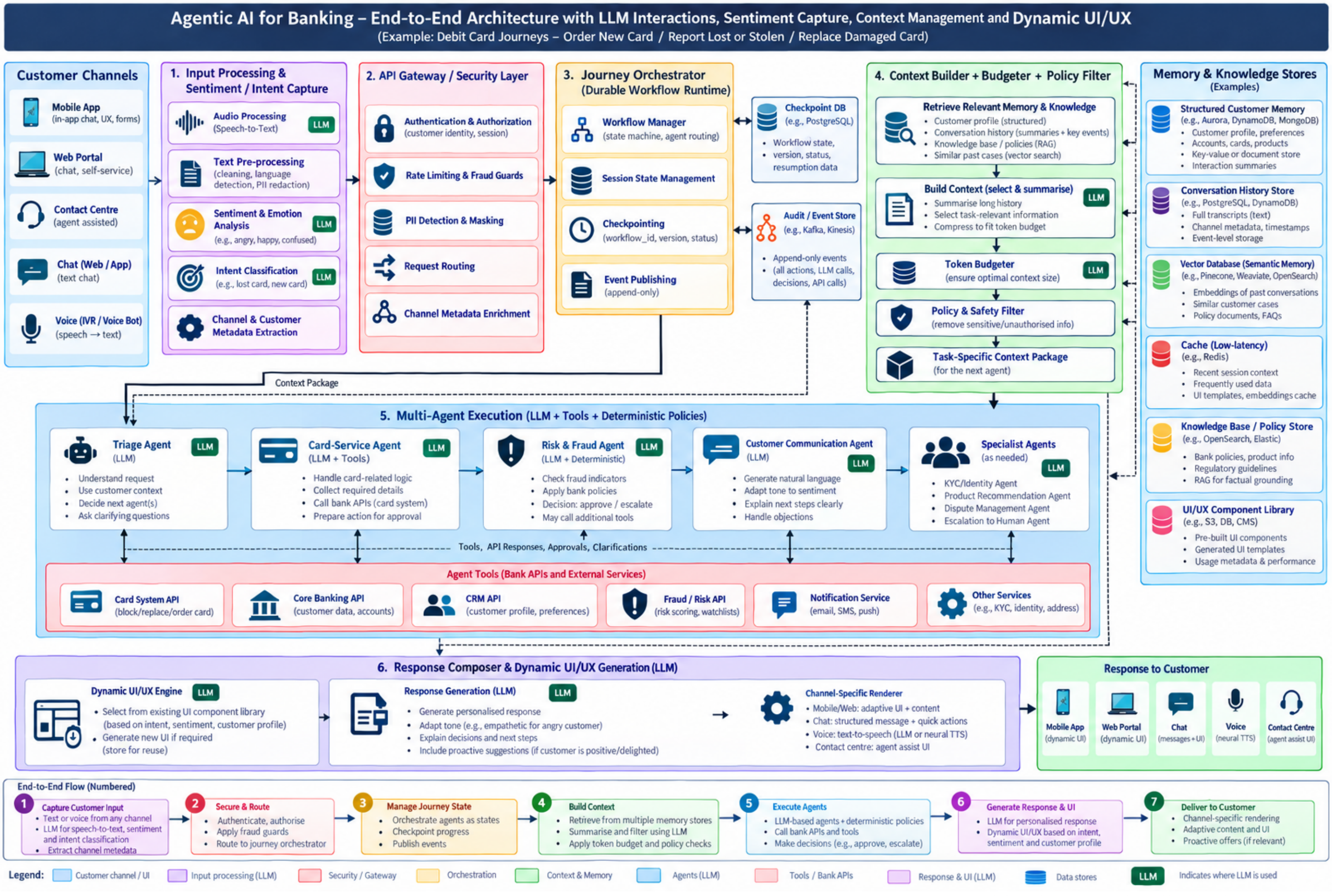
The reference architecture below brings these ideas together, showing the touchpoints for LLM calls, intent and sentiment detection, durable workflow state, multiple memory stores, specialist agents, bank-owned APIs and adaptive UI/UX.

Question for fellow architects: Would you build this as a multi-agent platform from the outset, or begin with a durable workflow and introduce specialist agents only where they demonstrably add value?

Technical references and further reading: APRA CPS 234 (https://www.apra.gov.au/standards/cps-234); APRA CPS 230 (https://www.apra.gov.au/consultations/operational-risk-management); OAIC APP 11 (https://www.oaic.gov.au/privacy/australian-privacy-principles/australian-privacy-principles-guidelines/chapter-11-app-11-security-of-personal-information); Microsoft Presidio (https://microsoft.github.io/presidio/analyzer/); LangGraph persistence (https://github.com/langchain-ai/docs/blob/main/src/oss/langgraph/persistence.mdx).

The Complete Reference Architecture

LLM touchpoints | intent and sentiment | orchestration | context | memory | adaptive UI/UX



Conceptual reference diagram. LLM markers show possible model calls; bank controls remain authoritative.